Overview

This project implements a fully serverless resume website hosted on AWS, with an automated backend and frontend deployment pipeline.

The system demonstrates:

- Static website hosting (S3 + CloudFront)
- Serverless backend (Lambda + API Gateway + DynamoDB)
- Infrastructure as Code (Terraform)
- CI/CD pipelines (GitHub Actions)
- Automated testing (pytest)

Summary

This project documents the end-to-end development of a cloud-hosted resume website built as part of the AWS Cloud Resume Challenge. The goal was not just to publish a static resume online, but to design and deploy a complete serverless web application using core AWS services and modern engineering practices. The final solution includes a static frontend hosted on Amazon S3 and delivered through CloudFront, a visitor counter powered by API Gateway, Lambda, and DynamoDB, infrastructure managed with Terraform, and automated backend and frontend deployment pipelines using GitHub Actions.

Beyond the final working system, this project also reflects the practical learning process involved in building real cloud solutions. Each phase required understanding how separate AWS services interact, how browser behavior affects frontend-backend communication, how infrastructure can be defined as code, and how automation can replace repetitive manual deployment steps. The result is not only a functioning resume website, but a reproducible, testable, and continuously deployable cloud project that demonstrates both technical implementation and problem-solving ability.

Architecture Summary

Frontend	❖ Hosted on S3 ❖ Distributed via CloudFront ❖ HTML, CSS and JavaScript
Backend	❖ API Gateway (HTTP API) ❖ Lambda (Python) ❖ DynamoDB (visitor counter)
DNS	❖ Managed using Route53 ❖ Custom domain: www.chong-teck-resume.com ❖ Routed traffic to CloudFront distribution
Infrastructure	❖ Managed using Terraform ❖ Remote state stored in S3 ❖ State locking via DynamoDB
CI/CD	❖ GitHub Actions ➢ Frontend (deploy + invalidate cache) ➢ Backend (test + deploy)

Phase 1 - Static Website Hosting

Objective:
Host a resume website using AWS with a custom domain.

Steps:

- Created static frontend files
  - app/index.html
  - app/style.css
  - app/returnVisitorCount.js
- Created S3 bucket
  - chongteck-crc
- Uploaded frontend files to S3
- Configured bucket policy
  - Public read access for static hosting
- Set up CloudFront distribution
  - Origin: S3 bucket
  - Enabled HTTPS delivery
- Purchased and configured domain using Route 53
  - Domain: https://chong-teck-resume.com/
  - Created hosted zone in Route 53
  - Added DNS records to point domain to CloudFront distribution
- Configured CloudFront
  - Added custom domain (www.chong-teck-resume.com)
  - Attached SSL certificate (via ACM)

Outcome:

- Resume website accessible via: https://chong-teck-resume.com/
- Secured with HTTPS
- Delivered globally CloudFront CDN

Phase 2 - Visitor Counter Backend

Objective:
Implement a dynamic visitor counter using serverless architecture.

Steps:
DynamoDB

1. Created table:
    a. Name: visitor-counter
    b. Partition key: id
    c. Attribute: count

Lambda Function

1. File: backend/retrieveVisitorCountPython.py
2. Logic:
    a. Increment visitor count
    b. Return updated count as JSON

API Gateway

1. Created HTTP API
2. Route: /count
3. Integrated with Lambda

Frontend Integration

1. JavaScript fetch call:
   https://qhnc6gyb59.execute-api.ap-southeast-1.amazonaws.com/prod/count
2. Updates DOM with visitor count

Outcome
Visitor counter updates dynamically on page load.

Phase 3 - CORS Configurations

Objective:
Allow frontend to backend API.

Implementation:

```
cors_configuration {
       allow_origins = ["*"]
       allow_methods = ["GET", "OPTIONS"]
       allow_headers = ["*"]
   }
```

Outcome:
Browser successfully performs cross-origin requests.

Phase 4 - Infrastructure as Code (Terraform)

Objective:
Replace manual AWS setup with reproducible infrastructure.

Key files:

- infra/main.tf
- infra/output.tf

Resources Managed:

- Lambda function
- API Gateway
- DynamoDB
- IAM roles and policies

Outcome:
Entire infrastructure defined as code and reproducible.

Phase 5 - Remote Terraform State

Objective:
Enable shared and consistent Terraform state.

Steps:

1. Bootstrap Infrastructure
   Created:
   - S3 bucket for state
   - DynamoDB table for state locking
     - Name: terraform-state-lock
     - Partition key: LockID

2. Backend Configuration

```
backend "s3" {
      # S3 bucket name for storing terraform state files
      bucket = "cloudresume-terraform-state-aaaa"
      # S3 key for the terraform state file
      key = "backend/terraform.tfstate"
      # aws region where the s3 bucket is located
      region = "ap-southeast-1"
      # DynamoDB table name for state locking
      dynamodb_table = "terraform-state-lock"
      # Enable encryption for the state file in S3
      encrypt = true
   }
```

3. Migration
   Ran "terraform init -migrate-state" to complete the migration of the state file in the S3 bucket for remote storage.

Outcome:

- State stored in S3
- Locking handled by DynamoDB
- Safe concurrent deployments

Phase 6 - Backend CI/CD

Objective:
Automate backend testing nd deployment.

File: .github/workflows/backend.yml

Workflow:

1. Checkout repository
2. Setup Python
3. Install dependencies (pytest, boto3)
4. Configure AWS credentials
5. Run tests
6. Terraform
    a. Init
    b. Plan
    c. Apply

Testing:
File: tests/test_lambda_function.py

This test verifies that the Lambda function:

- Executes successfully and returns a 200 status code
- Returns a valid JSON response
- Includes a visitor count field
- Ensures the count is an integer and non-negative

Key Issues Resolved:

- Python import path issues
- Missing **init**.py
- AWS region configuration
- AWS credential setup
- Terraform archive path fix

Outcome:
Backend is:

- Automatically tests
- Automatically deployed

Phase 7 - Frontend CI/CD

Objective:
Automate deployment of frontend assets.

File: .github/workflow/frontend.yml

Workflow:

1. Checkout repository
2. Configure AWS credentials
3. Sync files to S3:

```
- name: Upload frontend files to S3
    run: aws s3 sync app/ s3://chongteck-crc --delete
```

4. Invalidate CloudFront:

```
- name: Invalidate CloudFront cache
    run: aws cloudfront create-invalidation --distribution-id EHR6MMGY6G9CG --paths
"/*"
```

Outcome:

- Frontend updates automatically
- No manual upload required

Phase 8 - Debugging and Validation

Key Issues Encountered

1. CloudFront caching
    - Old JavaScript served
    - Fixed by creating invalidation to replace cache with new changes
2. Wrong API endpoint
    - Verified using browser developer tools, Network tab.
3. Terraform path issues
    - Fixed using ${path.module}
4. Python import errors
    - Fixed package structure (**init**.py in the same directory)
5. AWS credential errors
    - Fixed with correct key pair
6. DynamoDB access errors
    - Fixed IAM permisisons

Final System Flow

User visit website

↓

CloudFront serves frontend

↓

JavaScript calls API Gateway

↓

Lambda executes

↓

DynamoDB updates count

↓

Response returned

↓

Frontend displays count

Work Experience

Research and Development Engineer, Matwerkz Technologies
Aug 2023 — Present

Thermal Product Development

- Led high-temperature insulation composite development, improving heat resistance by 48% through material redesign and optimization.
- Streamlined testing and documentation for self-healing coatings, cutting validation time by four months.
- Achieved 80% UL94 V-0 pre-certification confidence via iterative evaluation and failure analysis.
- Collaborated with R&D and operations teams to apply test data to design decisions, ensuring product reliability.

AI and Data Systems Development

- Built multi-year R&D dataset with 400+ formulations and 7 metrics, developing Python ML workflows using pandas and scikit-learn for reproducible model testing.
- Diagnosed data imbalance and integrity issues through exploratory analysis, establishing validation rules that improved prediction reliability and overall model performance consistency.
- Prepared schema-consistent datasets for AWS S3/DynamoDB integration, collaborating with engineers to troubleshoot workflow errors and document corrective actions for scalability.

Education

BEng Mechanical Engineering [Honours with Merit] Oct 2023

Singapore Institute of Technology & University of Glasgow

- Specialization in Mechatronics

© 2025 Chong Teck. All rights reserved.

19 visitors have visited this resume.

© 2025 Chong Teck. All rights reserved.

19 visitors have visited this resume.

Summary and Future Improvements

This project successfully progressed from a manually configured cloud-hosted resume website into a more complete and production-style serverless application with automated deployment workflows. By the end of the challenge, the system supported a custom domain, HTTPS delivery, a live visitor counter, backend logic in Python, infrastructure defined in Terraform, remote Terraform state management, and separate CI/CD pipelines for backend and frontend updates. More importantly, the project provided hands-on experience in connecting cloud services into a working system and resolving real implementation issues along the way.

There are still several meaningful improvements that can be made in future iterations. Security can be strengthened by replacing static AWS access keys in GitHub Actions with OIDC-based authentication. The API configuration can be tightened by restricting CORS access to the production domain instead of allowing all origins. The backend pipeline can also be expanded with linting and formatting checks, while the frontend can be further improved with stronger deployment validation and cache control strategies. These improvements would make the project not only functional and automated, but also closer to the standard expected in real engineering environments.

Annex - Frontend

Index.html

```
<!DOCTYPE html>
<html>
   <head>
       <meta charset="utf-8">
       <title>
           Chong Teck's Resume
       </title>
       <link rel="stylesheet" href="style.css">
       <script src="returnVisitorCount.js" defer></script>
   </head>
   <body>
       <header>
           <h1>Resume</h1>
           <h1>Toh Chong Teck</h1>
           <p><a href="mailto:ctzongdetoh@gmail.com">ctzongdetoh@gmail.com</a>    |
83028649    |   <a href="www.linkedin.com/in/chong-teck-toh">LinkedIn</a>    |   <a
href="https://learn.nextwork.org/enthusiastic_blue_smart_rambutan/portfolio">Project
Portfolio</a></p>
       </header>

       <section id="about">
           <h2>About Me</h2>
           <p>My journey into cloud engineering started when my team needed to deploy
an AI application fast, but without spending on physical servers. That's when I
discovered how cloud computing could tackle both challenges at once, and I was hooked.
I went beyond my role and built a data pipeline connecting Amazon Aurora to an
EC2-hosted web application, letting researchers add data in real time. Seeing how the
right infrastructure helps teams move faster really clicked for me. I love that the
cloud lets you build, test, and improve quickly. I think the best solutions come from
trying things out and learning as you go, not getting everything perfect on the first
try. Now I'm working toward becoming a cloud engineer who creates infrastructure that
actually helps businesses get things done.</p>
       </section>

       <section id="skills">
           <h2>Skills</h2>
               <h3>Cloud and Infrastructure</h3>
               <ul>
                   <li>AWS EC2</li>
```

```
                <li>AWS S3</li>
                <li>AWS IAM</li>
                <li>AWS VPC</li>
                <li>AWS CloudFormation</li>
                <li>Terraform</li>
                <li>Cloud Security</li>
                <li>Cloud Networking</li>
                <li>Monitoring with VPC Flow Logs and CloudWatch</li>
                <li>Linux System Administration</li>
            </ul>
            <h3>Programming and Scripting</h3>
            <ul>
                <li>Python (pandas, scikit-learn, NumPy)</li>
                <li>Java (Basics)</li>
                <li>C++ (Basics)</li>
                <li>HTML/CSS</li>
                <li>Source Code Management (Git)</li>
            </ul>
            <h3>Data Engineering and ML</h3>
            <ul>
                <li>AWS DynamoDB</li>
                <li>AWS Amazon Aurora</li>
                <li>Data Cleaning, Validation and Feature Selection</li>
            </ul>
            <h3>System Operation</h3>
            <ul>
                <li>Technical Troubleshooting and Problem Solving</li>
                <li>Log Review and Performance Monitoring</li>
                <li>Technical Documentation and Stakeholder Management</li>
            </ul>
            <h3>Tools and Platform</h3>
            <ul>
                <li>AWS Console Management</li>
                <li>AWS CloudShell</li>
                <li>AWS QuickSight</li>
                <li>Tableau</li>
                <li>Microsoft Office</li>
            </ul>
        </section>

        <section id="certification">
            <h2>Certification</h2>
```

```
            <ul>
                <li>AWS Cloud Practitioner<br>Amazon Web Services (AWS)<br>Issued Aug
2025 — Expires Aug 2028<br><a
href="https://www.credly.com/badges/e259d5d3-17aa-497b-8c0a-5dcf1dfa0bde/linked_in_pro
file">Show Credential</a></li>
                <li>Pass the Technical Interview with Java Skill
Path<br>Codecademy<br>Issued Apr 2025<br>
<a
href="linkedin.com/redir/redirect/?url=https%3A%2F%2Fwww.codecademy.com%2Fprofiles%2Fc
tzongde%2Fcertificates%2F5f29ae1fc9647d0013a7284b&urlhash=HirC&isSdui=true&lipi=urn%3A
li%3Apage%3Ad_flagship3_profile_view_base_certifications_details%3BYXMC0ShjTC2qc3Ak54b
lGg%3D%3D">Show Credential</a></li>
                </li>
                <li>Learn Java Course<br>Codecademy<br>Issued Dec 2024<br><a
href="https://www.codecademy.com/profiles/ctzongde/certificates/d3f89367b558583e361640
f778191345">Show Credential</a></li>
                <li>Six Sigma Yellow Belt<br>Singapore Institute of
Technology<br>Issued Mar 2023<br>Credential ID: LSSYB-00119</li>
                <li>Python (Basic)<br>HackerRank<br>Issued Mar 2022<br><a
href="https://www.hackerrank.com/certificates/iframe/44f8e1fcf3f5">Show
Credential</a></li>
            </ul>
        </section>

        <section id="WorkExperience">
            <h2>Work Experience</h2>
            <h3>Research and Development Engineer, Matwerkz Technologies Aug 2023 —
Present</h3>
            <ul>
                <h4>Thermal Product Development</h4>
                <li>Led high-temperature insulation composite development, improving
heat resistance by 48% through material redesign and optimization.</lii>
                <li>Streamlined testing and documentation for self-healing coatings,
cutting validation time by four months.</li>
                <li>Achieved 80% UL94 V-0 pre-certification confidence via iterative
evaluation and failure analysis.</li>
                <li>Collaborated with R&D and operations teams to apply test data to
design decisions, ensuring product reliability.</li>

                <h4>AI and Data Systems Development</h4>
                <li>Built multi-year R&D dataset with 400+ formulations and 7 metrics,
developing Python ML workflows using pandas and scikit-learn for reproducible model
testing.</li>
```

```
                <li>Diagnosed data imbalance and integrity issues through exploratory
analysis, establishing validation rules that improved prediction reliability and
overall model performance consistency.</li>
                <li>Prepared schema-consistent datasets for AWS S3/DynamoDB
integration, collaborating with engineers to troubleshoot workflow errors and document
corrective actions for scalability.</li>
            </ul>
        </section>

        <section id="education">
            <h2>Education</h2>
            <h3>BEng Mechanical Engineering [Honours with Merit] Oct 2023</h3>
            <p>Singapore Institute of Technology & University of Glasgow</p>
            <ul>
                <li>Specialization in Mechatronics</li>
            </ul>
        </section>

        <footer>
            <p>© 2025 Chong Teck. All rights reserved.</p>
            <section id="visitorCount"></section>
        </footer>
    </body>
</html>
```

Style.css

```
/*
font: Claude
website color: #FCFCFC
side borders
centered title
*/

html, body {
   margin: 0;
   padding: 0;
   font-family: "Inter", -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
sans-serif;
   color: #1A1A1A;
   background-color: #FFFAEF;
}

header {
   text-align: center;
   padding: 40px;
   border-bottom: 2px;
   margin: 30px;
}

header h1 {
   font-size: 48px;
   margin: 0;
}

header p {
   font-size: 20px;
   margin: 10px 0 0 0;
}

h2 {
   font-size: 32px;
   margin-bottom: 20px;
   text-align: left;
}

h3 {
   font-size: 24px;
```

```
   margin-bottom: 15px;
   text-align: left;
}

#about {
   border-left: 2px;
   border-right: 2px;
   padding: 30px;
   max-width: 700px;
   margin: 0 auto;
   border-bottom: 2px solid #1A1A1A;
   margin-bottom: 30px;
}

#about p {
   font-size: 18px;
   line-height: 1.5;
}

#skills {
   border-left: 2px;
   border-right: 2px;
   padding: 30px;
   max-width: 700px;
   margin: 0 auto;
   border-bottom: 2px solid #1A1A1A;
   margin-bottom: 30px;
}

#skills ul {
   list-style-type: disc;
   padding-left: 20px;
   line-height: 1.5;
}

#certification {
   border-left: 2px;
   border-right: 2px;
   padding: 20px;
   max-width: 700px;
   margin: 0 auto;
   border-bottom: 2px solid #1A1A1A;
```

```
   margin-bottom: 30px;
}

#certification ul {
   list-style-type: disc;
   padding-left: 20px;
   line-height: 1.5;
}

#WorkExperience {
   border-left: 2px;
   border-right: 2px;
   padding: 20px;
   max-width: 700px;
   margin: 0 auto;
   border-bottom: 2px solid #1A1A1A;
   margin-bottom: 30px;
}

#WorkExperience ul {
   list-style-type: disc;
   padding-left: 20px;
   line-height: 1.5;
}

#education {
   border-left: 2px;
   border-right: 2px;
   padding: 20px;
   max-width: 700px;
   margin: 0 auto;
   border-bottom: 2px solid #1A1A1A;
   margin-bottom: 30px;
}

#education ul {
   list-style-type: disc;
   padding-left: 20px;
   line-height: 1.5;
}

h1, h2, h3 {
```

```
    color: #1A1A1A;
}
```

returnVisitorCount.js

```
console.log("Javascript file loaded: returnVisitorCounter.js");

const visitorCounter = document.getElementById("visitorCount");

// fetch the visitor count from the API invoke URL that would return a promise
fetch("https://qhnc6gyb59.execute-api.ap-southeast-1.amazonaws.com/prod/count")
// open the response as json
.then(Response => Response.json())
// set the innerHTML of the visitorCounter element to the count value from the json
response
.then(data => {visitorCounter.innerHTML = data.count + " visitors have visited this
resume.";
    console.log(data.count + " visitors have visited this resume.");})
// catch errors and log them to the console
.catch(error => console.error("Error fetching visitor count:", error));

console.log("This proves that the front CI/CD pipeline is working");
```

Annex - Backend

Lambda Function

[retrieveVisitorCountPython.py]

```
import json
import boto3

# iniitialize DynamoDB resource
dynamodb = boto3.resource("dynamodb", region_name="ap-southeast-1")
table = dynamodb.Table("visitor-counter")

# Function runs when Lambda is called
def lambda_handler(event, context):
    # increases the visitor count by 1
    response = table.update_item(
        # identify the item to update
        Key={"id": "resume"},
        # increment the count attribute by 1 or set it to 1 if it does not exist
        UpdateExpression="SET #count = if_not_exists(#count, :start) + :incr",
        ExpressionAttributeNames={"#count": "count"},
        ExpressionAttributeValues={":start": 0, ":incr": 1},
        ReturnValues="UPDATED_NEW"
    )

    new_count = int(response["Attributes"]["count"])

    return {
        'statusCode': 200,
        'body': json.dumps({
            "count": new_count
        })
    }
```

Annex - Infrastructure

infra/main.tf

```
terraform {
   # Terraform version constraint
   required_version = ">= 1.5.0"

   # Required providers
   required_providers {
       aws = {
           source = "hashicorp/aws"
           version = ">= 5.0.0"
       }
   }

   # Configure the backend to use AWS S3 for storing terraform state files and
DynamoDB for state locking
   backend "s3" {
       # S3 bucket name for storing terraform state files
       bucket = "cloudresume-terraform-state-aaaa"
       # S3 key for the terraform state file
       key = "backend/terraform.tfstate"
       # aws region where the s3 bucket is located
       region = "ap-southeast-1"
       # DynamoDB table name for state locking
       dynamodb_table = "terraform-state-lock"
       # Enable encryption for the state file in S3
       encrypt = true
   }

   # comment to test backend v11
}

# Configure the AWS provider
provider "aws" {
   # AWS region where the resources will be created
   region = "ap-southeast-1"
}

# Create a DynamoDB table for visitor counting
resource "aws_dynamodb_table" "visitor_counter" {
   # Name of DynamoDB table
```

```
   name = "visitor-counter"
   # Set billing mode to on demand (PAY_PER_REQUEST)
   billing_mode = "PAY_PER_REQUEST"
   # Define the primary key for the table
   hash_key = "id"

   # Define the attributes for the DynamoDB table
   attribute {
       # Name of the attribute
       name = "id"
       # Type of the attribute (String)
       type = "S"
   }
}

# Lambda IAM role to allow Lambda function to access DynamoDB
resource "aws_iam_role" "lambda_role" {
   # Name of the IAM role
   name = "lambda-role"
   # Assume role policy to allow Lambda service to assume this role
   # jsonencode is used to convert the policy into a JSON string format
   assume_role_policy = jsonencode({
       Version = "2012-10-17"
       Statement = [
           {
               Effect = "Allow",
               Principal = {
                   Service = "lambda.amazonaws.com"
               },
               Action = "sts:AssumeRole"
           }
       ]
   })
}

# Attach a policy to the IAM role to allow it to access the DynamoDB table
resource "aws_iam_role_policy" "lambda_policy" {
   # Name of the IAM role policy
   name = "lambda-policy"
   # The IAM role to attach this policy to
   role = aws_iam_role.lambda_role.id
```

```
   # Define the policy that allows the Lambda function to get and update items
   # jsonencode is used to convert the policy into a JSON string format
   policy = jsonencode({
       "Version" = "2012-10-17",
       "Statement" = [
           {
               "Effect" = "Allow",
               "Action" = [
                   "dynamodb:GetItem",
                   "dynamodb:UpdateItem"
               ],
               "Resource" = [
                   aws_dynamodb_table.visitor_counter.arn
               ]
           }
       ]
   })
}

# CloudWatch Logs IAM role policy to allow Lambda function to write logs to CloudWatch
resource "aws_iam_role_policy_attachment" "lambda_logs" {
   # Name of IAM role policy attachment
   role = aws_iam_role.lambda_role.name
   # ARN of the AWS managed policy for Lambda basic execution role, which allows
writing logs to CloudWatch
   policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

# Package Lambda function code into a zip file
data "archive_file" "visitor_counter_zip" {
   # type of archive set to zip
   type = "zip"
   # file path to the lambda function source code
   source_file = "${path.module}/../backend/retrieveVisitorCountPython.py"
   # file path to the output zip file
   output_path = "${path.module}/retrieveVisitorCountPython.zip"
}

# Create a Lambda function to handle visitor counting
resource "aws_lambda_function" "visitor_counter_function" {
   function_name = "visitor-counter-function"
```

```
   # Lambda IAM role to allow the function to access DynamoDB and write logs to
CloudWatch
   role = aws_iam_role.lambda_role.arn

   # Run time environment for the Lambda function
   runtime = "python3.14"

   # Lambda function handler: file name and function name within the file
   handler = "retrieveVisitorCountPython.lambda_handler"

   # file path to the zip file containing the Lambda function code
   filename = data.archive_file.visitor_counter_zip.output_path

   # source code hash to ensure Lambda function is updated when the source code
changes
   source_code_hash = data.archive_file.visitor_counter_zip.output_base64sha256

   # Ensure that the Lambda function is created after the IAM role and policies are in
place
   depends_on = [
       aws_iam_role_policy.lambda_policy,
       aws_iam_role_policy_attachment.lambda_logs
   ]
}

# API Gateway to trigger the Lambda function when the website is accessed
# HTTP API > Integration > Route > Stage > Deployment
# Create an API Gateway HTTP API
resource "aws_apigatewayv2_api" "visitor_counter_api"{
   name = "visitor-counter-api"
   protocol_type = "HTTP"

   # CORS configuration to allow cross-origin requests from the website to the API
Gateway endpoint
   cors_configuration {
       allow_origins = ["*"]
       allow_methods = ["GET", "OPTIONS"]
       allow_headers = ["*"]
   }
}

# Create an integration between API Gateway and the Lambda function
```

```
resource "aws_apigatewayv2_integration" "visitor_counter_integration" {
   # the API Gateway to integrate with
   api_id = aws_apigatewayv2_api.visitor_counter_api.id
   # type of integration set to AWS_PROXY to allow API Gateway to proxy requests
directly to the Lambda function
   integration_type = "AWS_PROXY"
   # the Lambda function to integrate with
   integration_uri = aws_lambda_function.visitor_counter_function.arn
   # expects the Lambda function to return a response in the format expected by API
Gateway
   payload_format_version = "2.0"
}

# Create a route in API Gateway to trigger the Lambda function when the root path is
accessed
resource "aws_apigatewayv2_route" "visior_counter_route" {
   # the API Gateway to create the route for
   api_id = aws_apigatewayv2_api.visitor_counter_api.id
   # the route key defines the HTTP method and path that will trigger this route.
   route_key = "GET /count"
   # the integration to trigger when this route is accessed
   target =
"integrations/${aws_apigatewayv2_integration.visitor_counter_integration.id}"
}

# stage to deploy the API Gateway
resource "aws_apigatewayv2_stage" "visitor_counter_stage" {
   # the API Gateway ID to create the stage for
   api_id = aws_apigatewayv2_api.visitor_counter_api.id
   # name of the stage
   name = "prod"
   # auto deploy changes to the stage when the API Gateway configuration changes
   auto_deploy = true
}

# "aws_lambda_permission" resource to allow API Gateway to invoke the Lambda function
resource "aws_lambda_permission" "apigw_lambda_permission" {
   # the Lambda function to allow API Gateway to invoke
   function_name = aws_lambda_function.visitor_counter_function.function_name
   # a unique identifier for this permission statement
   statement_id = "AllowAPIGatewayInvoke"
   # the action that API Gateway is allowed to perform on the Lambda function
```

```
   action = "lambda:InvokeFunction"
   # the principal that is allowed to invoke the Lambda function, which is the API
Gateway service
   principal = "apigateway.amazonaws.com"
   # the source ARN specifies which API Gateway routes are allowed to invoke the
Lambda function
   source_arn = "${aws_apigatewayv2_api.visitor_counter_api.execution_arn}/*/*"
}
```

infra/output.tf

```
output "visitor_counter_api_url" {
   # Output the URL of the API Gateway endpoint to access the visitor counter API
   # The argument value is constructed using the API endpoint from the
aws_apigatewayv2_api resource and appending the /count route_key defined in the
aws_apigatewayv2_route resource
   value =
"${aws_apigatewayv2_api.visitor_counter_api.api_endpoint}/${aws_apigatewayv2_stage.vis
itor_counter_stage.name}/count"
}
```

infra/bootstrap/main.tf

```
# Provision of AWS S3 bucket and DynamoDB for storing terraform state files to prevent
state file corruption and enable collaboration among team members

terraform {
   required_version = ">=1.5.0"
   required_providers {
       aws = {
           source = "hashicorp/aws"
           version = "~> 5.0"
       }
   }
}

provider "aws" {
   region = "ap-southeast-1"
}

resource "aws_s3_bucket" "terraform_state" {
   # S3 bucket name for storing terraform state files
   bucket = "cloudresume-terraform-state-aaaa"
}

resource "aws_s3_bucket_versioning" "terraform_state_versioning" {
   bucket = aws_s3_bucket.terraform_state.id

   versioning_configuration {
       status = "Enabled"
   }
}

resource "aws_dynamodb_table" "terraform_lock" {
   # DynamoDB table name for state locking
   name = "terraform-state-lock"
   # Set billing mode to on demand
   billing_mode = "PAY_PER_REQUEST"
   # Define the primary key for the table
   hash_key = "LockID"

   attribute {
       # Name of the attribute
       name = "LockID"
```

```
        # Type of attribute (String)
        type = "S"
    }
}
```

Annex - Test

test_lambda_function.py

```
# import json module to convert JSON text to a Python object
import json
# import os and sys modules to manipulate the Python path for importing the
lambda_handler function from the backend directory
import os
import sys

# add parent directory of the current file to the Python path to allow importing the
lambda_handler function from the backend directory
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

# go to the file named retrieveVisitorCountPython.py and import the lambda_handler
function
from backend.retrieveVisitorCountPython import lambda_handler

# test function to check if the lambda_handler returns a valid count
def test_lambda_returns_count():
   # call the lambda_handler function with empty event and context
   response = lambda_handler({}, None)

   # checks if response status code is 200
   assert response["statusCode"] == 200

   # parse the body of the response as JSON
   body = json.loads(response ["body"])

   # checks if the body contains a "count" key and that its value is an integer
   assert "count" in body
   # check if the count is an integer
   assert isinstance(body["count"], int)
   # check if the count is non-negative
   assert body["count"] >= 0
```